

A small horizontal bar with a teal segment on the left and an orange segment on the right.

CSC 84040: Advanced Data Mining *with AI Agents*

Lecture 03: Similarity Search

By Saptarashmi Bandyopadhyay

Email: sbandyopadhyay@ccny.cuny.edu

Assistant Professor of Computer Science

City College of New York and Graduate Center at the City University of New York

September 16, 2026 CSC 84040

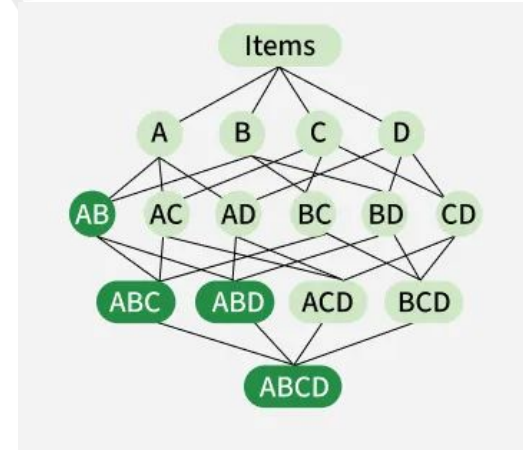
The Apriori Algorithm

—

Introduction

The A-Priori Algorithm exploits the monotonicity property of itemsets to constrain main memory usage.

It only generates candidate itemsets of size k if all their subsets of size $k - 1$ are previously proven to be frequent.





The First Pass (Apriori)

On the first pass, we scan the dataset to get the first count of each item

We build a translation table in-memory to map item indices from 1 to n

We count the occurrences in an array and find which items meet the predetermined support threshold s .

Items surpass (or meet) this support threshold s are frequent items.

We generate a new numbering system that assigns integers 1 to m to frequent items, which vastly compresses our needed memory footprint.



Second Pass

The second pass focuses solely on candidate pairs formed by the subset of frequent items identified in the first pass.

It scans the baskets again, evaluating possible pairs for each basket only if both items in the pair are individually frequent.

Occurrences of these filtered candidate pairs are aggregated to find the true frequent pairs.



Apriori: Generalization to Size k

The algorithm creates a set of candidate itemsets C_k and scans the baskets to count their support. Any itemsets failing to satisfy the support threshold are pruned before iterating to $k + 1$.

Unfortunately, Apriori fundamentally requires generating all candidate combinations of items.

It requires multiple full scans of the transactional database, which degrades computational speed on massive datasets.

Example problem: <https://www.geeksforgeeks.org/machine-learning/apriori-algorithm/>

FP-Growth

—



Introduction

FP-Growth = Frequent Pattern Growth: We mine the frequent datasets, but we don't need to take on the cost of generating candidates.

Instead, we compress the dataset into a more compact FP-Tree.



FP-Growth Item Frequency and Sorting

The algorithm first scans the entire dataset to compute the exact frequencies of individual items.

It filters out infrequent items and sorts the items in every individual transaction in descending order based on their overall frequency in the dataset.

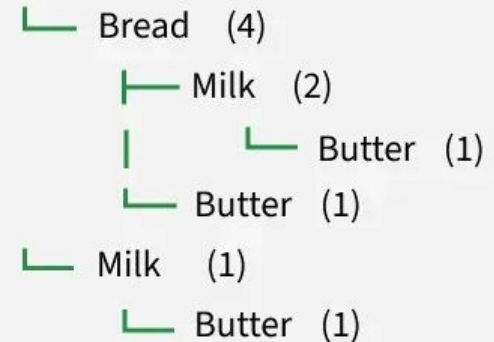
Transaction ID	Sorted Items by Frequency
T1	Bread, Milk, Butter
T2	Bread , Butter
T3	Bread, Milk
T4	Milk , Butter
T5	Bread

Constructing the FP-Tree

The sorted transactions are inserted iteratively into the FP-tree structure.

During insertion, shared prefixes among different transactions are merged, which compactly stores the data and drastically minimizes redundancy.

Root





Conditional Pattern Bases

After building the main FP-tree, the algorithm determines a "conditional pattern base" for each item.

The conditional pattern base consists of the specific prefix paths within the FP-tree that co-occur with the target item.

[(Bread, Milk): 1, (Bread): 1, (Milk): 1]



Constructing Conditional FP-Trees

We construct a smaller, more localized FP-tree for each particular item using the extracted conditional pattern base.

This way allows us to efficiently find frequent patterns by isolating the search space to the specific item.

Since both meet the support threshold (≥ 2), we can now generate frequent patterns:

- {Butter, Bread}; {Butter, Milk}; {Butter, Bread, Milk}

(Bread, Milk): 1

(Bread): 1

(Milk): 1

Count all items:

Bread: 2

Milk: 2

Mining the Tree Recursively



Pattern Extraction

The algorithm examines these localized conditional trees to extract patterns that independently satisfy the minimum support threshold.



Recursive Breakdown

This approach recursively breaks down the primary tree into smaller structures, mining deeper itemsets efficiently.



Efficiency Advantages of FP-Growth

The FP-Growth algorithm strictly scans the database only two times regardless of maximum pattern length.

By completely bypassing the candidate generation step, it mitigates the exponential search space bottleneck seen in Apriori.

It is highly scalable and executes exceptionally well for large, dense datasets.



Limitations of FP-Growth

Memory consumption for storing the FP-tree can become exceptionally high, as the tree structure scales extensively with highly diverse datasets.

The complex recursive logic and tree implementations make it significantly more difficult to build and interpret compared to the Apriori algorithm.

Similarity Search for Data Mining

—



Managing high-d memory in Agentic systems

As generative text agents interact over weeks, episodic logs, tool traces, and document caches explode.

We cannot keep raw interaction history in context windows without severe latency, memory bloat, and attentional dilution.

Today (time allowing), we will be covering:

- Set-theoretic similarity:
 - Shingling
 - Jaccard distance
 - Minhash signatures.



Avoiding Memory Pollution (A POMPDP Perspective)

Memory M_t serves as the agent's internal belief state summarizing history H_t :

$$a_t = \pi_\theta(x_t, \mathcal{R}(M_t, x_t), g_t), \quad M_{t+1} = \mathcal{U}(M_t, x_t, a_t, o_t, r_t)$$

where $\mathcal{R}$ is the retrieval operator and $\mathcal{U}$ is the memory update policy.

Problems resulting from Unbounded History:

- Attentional Dilution: "Lost in the middle" degradation as context length increases.
- Summarization Drift: Repeated recursive compression washes out critical low-frequency constraints.
- Memory Bloat: Repeated outputs and near-identical interaction traces saturate retrieval indexes, returning redundant distractors to the prompt.

We need sub-linear similarity search to detect near-duplicate episodes on the write path and retrieve relevant memories on the read path.

Fundamentals of Set Similarity

Jaccard Similarity: For two sets $S, T \subseteq U$:

$$SIM(S, T) = \frac{|S \cap T|}{|S \cup T|}$$

Jaccard Distance: Defined as

$$d_J(S, T) = 1 - SIM(S, T)$$

Metric Axiom Verification for d_J :

- Non-negativity: $d_J(S, T) \geq 0$ (since $|S \cap T| \leq |S \cup T|$).
- Identity of indiscernibles: $d_J(S, T) = 0 \iff S = T$
- Symmetry: $d_J(S, T) = d_J(T, S)$
- Triangle Inequality: $d_J(S, T) \leq d_J(S, R) + d_J(R, T)$ (proven via set indicator vectors).

Example Applications:
Comparing sets of visited web states, tool-argument bags, or overlapping document chunks without relying on dense neural embeddings



Shingling Text Traces and Observations

k -Shingles: A document or execution log is mapped to a set of substrings of length k .

Choosing k :

- If alphabet size is $|A|$, total possible shingles is $|A|^k$.
- Small k : High probability that unrelated logs share shingles by chance.
- For English logs/emails: $k = 5$ or $k = 9$ is standard to suppress background collisions.
- Hashed Shingles: Map k -character strings to 32-bit integers via hash $h : \text{string} \rightarrow [0, 2^{32} - 1]$, compressing storage to 4 bytes per unique shingle while retaining set identity.



Quadratic Pairwise Complexity Problems

Given N agent memories or documents, finding all pairs with $SIM(S_i, S_j) \geq t$ naively requires:

$$\binom{N}{2} = \frac{N(N-1)}{2} = O(N^2) \text{ comparisons}$$

Scale Breakdown:

- $N = 10^5$ episodes $\implies \approx 5 \times 10^9$ distance computations.
- $N = 10^6$ episodes $\implies \approx 5 \times 10^{11}$ distance computations.
- High-dimensional vector dot products or large set intersections across terabytes of agent logs will stall any real-time agent execution loop.

we desire an algorithmic filter that projects N objects into buckets such that only candidate pairs with high prior probability of similarity are compared.



Characteristic Matrix

Characteristic Matrix M:

- Rows = universal set of all elements (e.g., all observed shingles).
- Columns = individual sets / episodic records.

$M_{r,c} = 1$ if element $r \in \text{Set } c$, else 0.

- Highly sparse in practice.



Minhashing

Minhashing Concept:

- Let π be a random permutation of the rows of M .
- The minhash function $h_\pi(S)$ is the index of the first row in permuted order where column S contains a 1:

$$h_\pi(S) = \min\{\pi(r) \mid r \in S\}$$

Theorem: The probability that two sets have the same minhash value under a random permutation equals their Jaccard similarity:

$$P[h_\pi(S_1) = h_\pi(S_2)] = \text{SIM}(S_1, S_2)$$



Proof Sketch

Row Partitioning:

- Type X: Row has 1 in both columns S_1 and S_2 ($|S_1 \cap S_2| = x$).
- Type Y: Row has 1 in one column and 0 in the other ($|S_1 \cup S_2| - |S_1 \cap S_2| = y$).
- Type Z: Row has 0 in both columns (irrelevant to $S_1 \cup S_2$).

Derivation:

- Scan downward in permuted order π .
- Type Z rows provide no information.
- The first non-Z row encountered is either Type X or Type Y.
- $h(S_1) = h(S_2)$ if and only if the first non-Z row is of Type X.

Therefore:

$$P[h_\pi(S_1) = h_\pi(S_2)] = \frac{x}{x+y} = \frac{|S_1 \cap S_2|}{|S_1 \cup S_2|} = SIM(S_1, S_2)$$



Minhashing in Practice (part 1)

Physical Permutations are Impossible: Explicitly shuffling a matrix with millions of rows is computationally prohibitive.

Hash Function Simulation:

- Pick n independent linear hash functions:

$$h_i(r) = (a_i \cdot r + b_i) \pmod{p}$$

where p is a prime larger than the number of universal shingles, and a_i, b_i are random integers.



Minhashing in Practice

Signature Matrix Computation:

- Initialize signature matrix $SIG(i, c) = \infty$ for all hash functions $i \in [1, n]$ and columns c .
- For each row r with elements:
 - Compute $h_1(r), h_2(r), \dots, h_n(r)$
 - For each column c where $M_{r,c} = 1$:

$$SIG(i, c) \leftarrow \min(SIG(i, c), h_i(r)) \quad \forall i \in [1, n]$$

Result: Compresses a sparse column of size $|U|$ down to a dense vector of n integers; typically $n = 100$ to 256

Questions?

—